

PROJETO: Aplicativo Desktop – Assinatura de jogos educacionais (Messier)

Disciplina: Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO E RAs:

- Lucas Kazuo Dos Santos Kanegae - 26029179
- Pedro Henrique Costa Barros Santos - 25027375
- Vitor Ordonis Henrique - 26028722
- Henrique Bezerra Nunes - 26029106

São Paulo, 2025

SUMÁRIO

1. INTRODUÇÃO	3
1.1 – Processo de desenvolvimento de Software	3
 2. DESIGN SPRINT – Ideação e prototipação do desafio	4
2.1 Desafio	4
2.2 Desenho da Solução	4
2.3 Prototipagem	5
 3. REQUISITOS DE SISTEMA	6
3.1 REQUISITOS FUNCIONAIS DE SOFTWARE	6
3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE	7
 4. CASOS DE USO	8
4.1 Caso de uso: administrador	10
4.2 Caso de uso: Escola.....	11
 5. ARQUITETURA DO SISTEMA	9
5.1 Visão Geral do Sistema.....	10
5.2 Estrutura Geral do Sistema.....	11
5.3 Tecnologias Utilizadas.....	12
5.4 Componentes do Sistema	13
5.5 Diagrama simplificado.....	10
5.2 Organização de Arquivos e Pastas.....	11

1. APRESENTAÇÃO

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para gerar engajamento em eventos, lançamentos de produtos e campanhas de marca. Como novo projeto, a empresa quer lançar uma plataforma de jogos digitais voltada para escolas do ensino fundamental e médio.

Neste Projeto Interdisciplinar, nossa equipe irá desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas. Com ele, será possível controlar os pacotes contratados, os limites de acesso mensais, os IPs autorizados e os relatórios de uso.

Simulação do módulo Escola:

1. **Login:** a escola entra no sistema com suas credenciais e o aplicativo verifica se o pacote está ativo e se o IP informado está autorizado.
2. **Catálogo:** depois do login, a escola visualiza os jogos disponíveis no seu pacote.
3. **Acesso:** a escola escolhe um jogo para acessar e o sistema registra esse acesso no consumo do mês, respeitando o limite do pacote contratado.

Acesso de administrador:

Permite o acesso a relatórios que demonstram o histórico de acessos, visualização da quantidade de acessos disponíveis em cada assinatura, controle das tentativas de acesso negadas e registro, atualização e exclusão de dados cadastrados.

1.1 – Processo de desenvolvimento de Software

O projeto seguiu o framework Scrum adaptado ao contexto acadêmico, com um time de 5 pessoas nos papéis de Product Owner, Scrum Master e Desenvolvedores. Henrique Bezerra atuou como Product Owner, sendo responsável por definir e priorizar os requisitos, enquanto Pedro Henrique exerceu o papel de Scrum Master, facilitando o processo. A equipe de desenvolvimento foi feita por Lucas, Giovanna, Pedro, Vitor e Henrique, responsáveis pela implementação do produto.

O desenvolvimento foi organizado em 4 Sprints de 1 semana cada. Sprints são ciclos curtos e iterativos de desenvolvimento, característicos do framework Scrum, nos quais o time trabalha de forma focada para entregar um incremento funcional do produto ao final de cada período. Em cada Sprint, o time seguiu o fluxo de Backlog, Planejamento, Desenvolvimento, Testes, Entrega e Retrospectiva.



2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

As equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo.

2.2 Desenho da solução

Principal user: Aluno

- **Passo 1 – Login e Autenticação:** O aluno insere login e senha. O sistema verifica pacote, IP e credenciais. Caso inválido, exibe mensagem de bloqueio.
- **Passo 2 – Aplicação Principal:** Visualização do painel com informações da escola e jogos recentes.

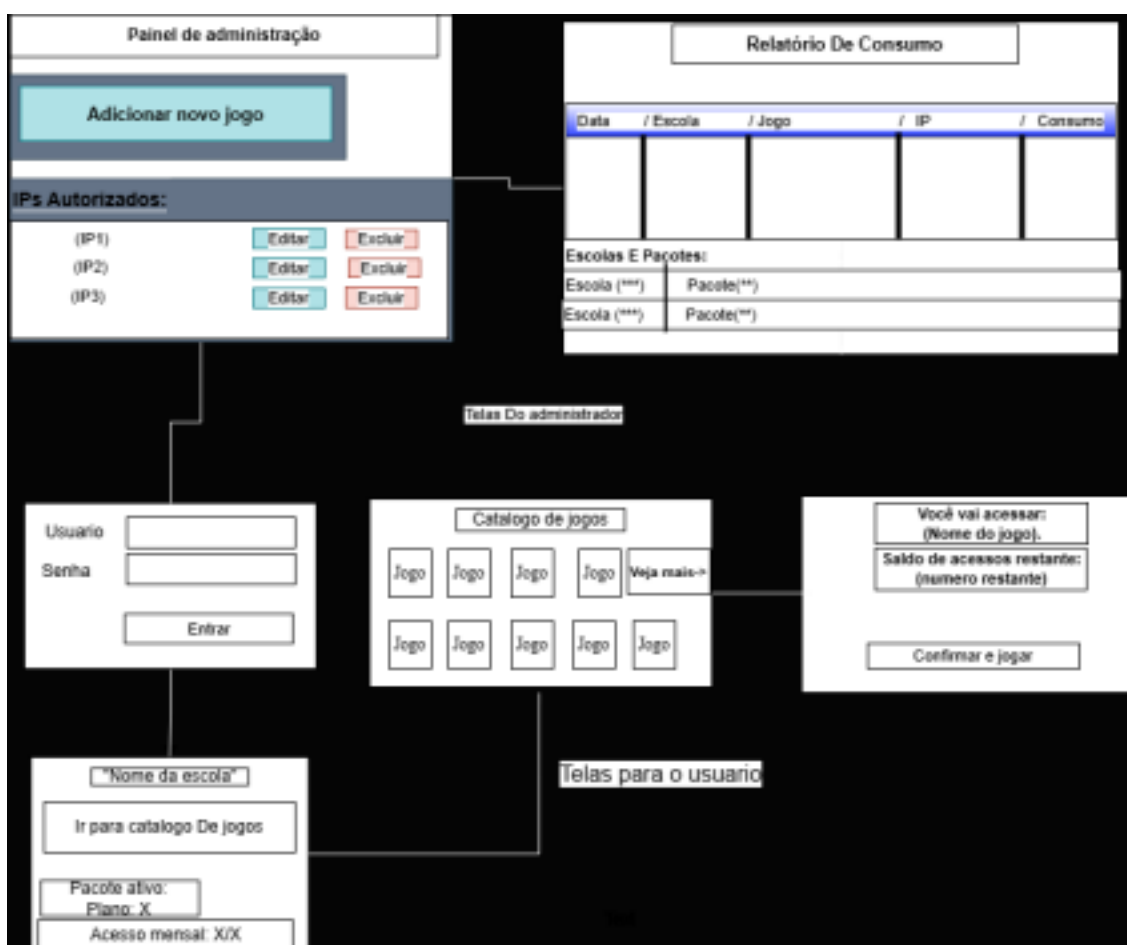
- **Passo 3 – Catálogo de Jogos:** Visualização de nome, descrição, tema e faixa etária.
- **Passo 4 – Acesso ao Jogo:** O sistema computa o uso ao selecionar o jogo.
- **Passo 5 – Finalização:** Retorno à tela principal após encerrar.

Trilha do administrador:

- **Passo 1 – Login Administrador:** Acesso via credenciais administrativas.
- **Passo 2 – Dashboard:** Cadastro, edição ou remoção de escolas, pacotes e IPs.
- **Passo 3 – Gerenciamento:** Definição de jogos por pacote, limites e preços.
- **Passo 4 – Monitoramento:** Acompanhamento de relatórios de consumo e histórico.

2.3 Prototipagem

- **Tela 1:** Login e mensagens de erro (IP/Plano).
- **Tela 2:** Painel principal com alertas de limite de acesso.
- **Tela 3:** Catálogo de jogos conforme o pacote.
- **Tela 4:** Confirmação de acesso e saldo restante.
- **Tela 5:** Dashboard administrativo e gestão de IPs.
- **Tela 6:** Tabela de histórico de consumo e relatórios por período.



3. REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE (RFS)

ID	Função	Descrição	Entradas	Fonte	Saídas	Ação	Prioridade	História de Usuário
RFS 01	Cadastro de jogos	Permite cadastrar jogos na plataforma.	Nome, descrição, disciplina, faixa etária e status.	Admin	Jogo cadastrado e disponível.	Validar dados e armazenar no banco.	Alta	Como admin, quero cadastrar jogos para que fiquem disponíveis nos pacotes.
RFS 02	Bloquear acesso por limite	Impede novos acessos quando o limite é atingido.	Tentativa de acesso e contador mensal.	Sistema	Acesso bloqueado e mensagem explicativa.	Comparar contador com limite e bloquear se necessário.	Alta	Como sistema, quero bloquear acessos excessivos para respeitar o contrato.
RFS 03	Cadastro de escolas	Gerenciar escolas, pacotes e IPs autorizados	Nome, endereço, telefone, pacotes	Admin	Confirmação de cadastro e escola vinculada.	Validar um pacote por escola e vincular	Alta	Como admin, quero gerenciar escolas para configurar

		s.	e IPs.			IPs.		o acesso corretamente.
RFS 04	Login Módulo Escola	Validar acesso via credenciais e IP.	Usuário, senha e IP de origem.	Coordenador	Acesso concedido ou erro.	Verificar credenciais, pacote ativo e IP autorizado.	Alta	Como coordenador, quero logar para garantir que apenas acessos válidos entrem.
RFS 05	Catálogo de jogos	Exibir jogos do pacote ativo.	Dados da escola e pacotes.	Sistema	Lista de jogos com informações completas.	Consultar pacote ativo e listar jogos vinculados.	Média	Como aluno, quero ver apenas meus jogos para escolher o que acessar.
RFS 06	Registrar acesso	Atualizar contador e log de uso.	Jogo, dados da escola, IP e data.	Sistema	Log registrado e contador incrementado.	Capturar dados, validar acesso e atualizar banco.	Alta	Como sistema, quero registrar acessos para fins de auditoria e controle.

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE (RNFS)

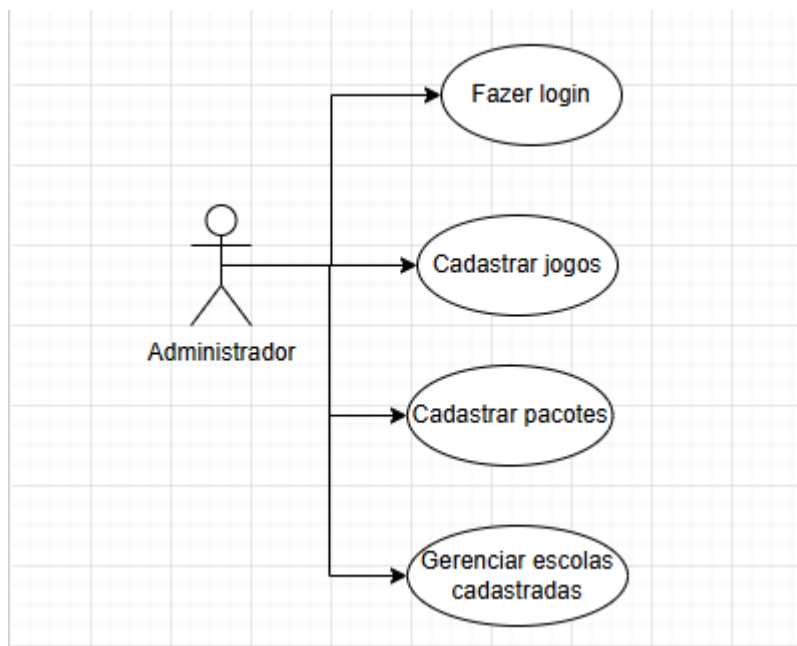
ID	Função	Descrição

RNFS01	Funcionamento Offline	A aplicação deve operar de forma local para fins de protótipo, sem depender de internet.
RNFS02	Interface Intuitiva	Deve ser compatível com Windows Desktop, usando padrões de listas e formulários.
RNFS03	Tempo de Resposta	Consultas e relatórios devem ser rápidos, sem travamentos em bases médias.
RNFS04	Código em Camadas	Organização separada entre Interface, Regras de Negócio e Persistência.
RNFS05	Integridade de Dados	Uso de banco relacional com chaves e relacionamentos para evitar duplicidade.
RNFS06	Documentação	O projeto deve conter README e manual rápido de execução padronizados.

4. CASOS DE USO

4.1 Caso de Uso: Administrador

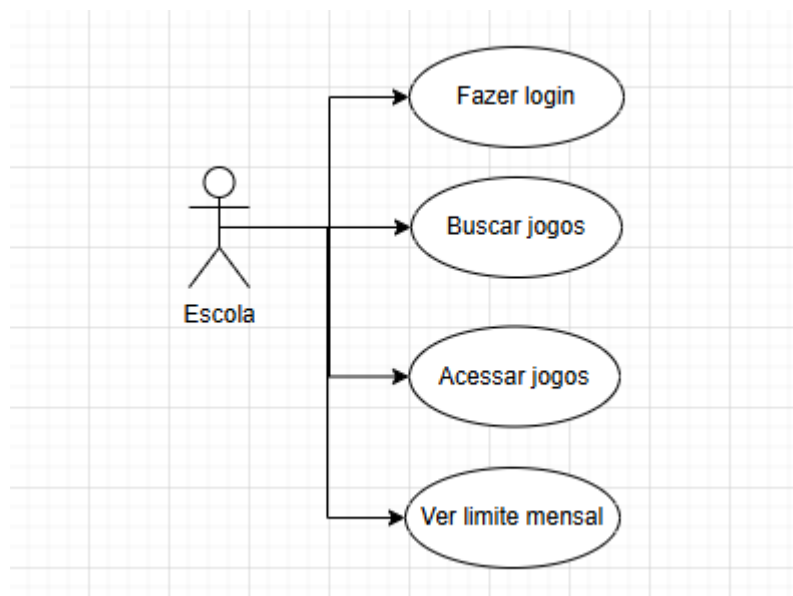
Esta seção detalha as interações do Ator Administrador, responsável pela gestão técnica e operacional da plataforma Messier.



Campo	Descrição
Ator	Administrador
Objetivo	Gerenciar o ecossistema da plataforma (jogos, pacotes e escolas).
Entrada	Credenciais administrativas, dados dos jogos, definições de pacotes e informações das escolas.
Saída	Sistema atualizado com novos registros, alterações ou exclusões de dados cadastrais.

4.2 Caso de Uso: Escola

Esta seção detalha as interações do Ator Escola (Coordenador/Aluno), focado no consumo dos conteúdos e monitoramento de uso.



Campo	Descrição
Ator	Escola (Coordenador ou Aluno)
Objetivo	Acessar e utilizar os jogos educacionais dentro dos limites contratuais.
Entrada	Credenciais da escola, IP de origem e seleção de jogos no catálogo.
Saída	Acesso concedido aos jogos e visualização do status de consumo mensal.

5. ARQUITETURA DO SISTEMA

5.1 Visão Geral do Sistema

O Sistema Messier (Grupo 1) foi projetado para oferecer uma estrutura robusta e organizada, abrangendo todas as camadas necessárias para a gestão de jogos educacionais. O objetivo central é fornecer às Escolas parceiras da Messier uma plataforma segura e eficiente.

- **Principais Funcionalidades:** Catálogo de jogos personalizado, controle de planos de assinatura e monitoramento de limites mensais de acesso.

5.2 Estrutura Geral do Sistema

A arquitetura baseia-se em uma aplicação Desktop Windows para interação direta com o usuário. As regras de negócio são implementadas em C# (com lógica validada via pseudocódigo) e o armazenamento de dados utiliza o banco de dados MySQL, garantindo persistência e integridade das informações.

5.3 Tecnologias Utilizadas

- **Linguagem de Programação:** C#
- **Modelagem de Dados:** BRModelo
- **Lógica de Programação:** Visualg (Pseudocódigo)
- **Banco de Dados:** MySQL
- **Controle de Versão:** GitHub

5.4 Componentes do Sistema

A tabela abaixo descreve as responsabilidades de cada módulo principal do sistema:

Componente	Responsabilidade
Cadastro de Jogos	Gerenciar o acervo de games, incluindo metadados como disciplina e faixa etária.
Cadastro de Escolas	Registrar instituições parceiras e seus respectivos endereços de IP autorizados.

Cadastro de Pacotes	Definir as regras de negócio, limites de acessos mensais e jogos inclusos em cada plano.
----------------------------	--

5.5 Diagrama Simplificado

O fluxo de dados do sistema segue a hierarquia abaixo:

1. **Usuário:** Interage com as interfaces.
2. **Telas do Sistema:** Camada de interface (Windows Forms).
3. **Lógica de Aplicação:** Processamento das regras de negócio (C#).
4. **Banco de Dados:** Armazenamento persistente (MySQL).

5.6 Organização de Arquivos e Pastas

Para garantir a rastreabilidade e organização acadêmica, o projeto está estruturado em diretórios específicos:

- **Estrutura:** Uma pasta dedicada para cada disciplina do semestre, contendo suas respectivas entregas, documentações e versões do código-fonte.